Code Jam 2020 - Round 1B

Analysis: Blindfolded Bullseye

Test Set 1

In Test Set 1 the radius is both really large and known in advance. The fact that it is large gives us a major restriction: plugging in the value for R in the limits for X and Y shows that the hidden center is restricted to a square with side length 10 nanometers, centered on the origin. That means that there are only $11^2 = 121$ possible positions for the center! We can simply try throwing a dart at each of them, stopping as soon as we are told that we have hit the `CENTER`.

Test Set 2

In Test Set 2 the radius is also large and known, but small enough that the square of candidate center points has side 101 nanometers. This means that there are 10201 possible centers, which is a lot more than the number of darts we get. We need to restrict that further.

We can use the fact that we know the exact radius to reduce this number. Every dart we throw can give us a clue. If a dart thrown at point p hits, that means the the distance between p and the center c is no more than R. If a dart thrown at point q misses, that means that the distance between q and c is more than R. If we find two points p and q that are close to each other, the points that are no more than R away from p but more than R away from q form a narrow crescent-shaped region of width equal to the distance between p and q. Intersecting that with the original square of possibilities from the first paragraph can give us a small enough range for c that we can just throw a dart at each possibility without running out of darts.

We can find points p and q that are close to each other by finding the edge of the dartboard. Since the dartboard is so large, the edge has to be close to the edge of the wall. If we inspect points (x, 0) for the possible values of x, most points are guaranteed to be inside the dartboard: any point (x, 0) with $-10^9 + 101 \leq x \leq 10^9 - 101$ is no more than R away from all possible centers, and is thus guaranteed to be inside the circle. Therefore, we can try all possible x on any one side of that guaranteed interval to find a hit point and a miss point that are 1 nanometer away from each other. Since there are only 101 points on each side that are not guaranteed to be in the circle — for example $[-10^9, -10^9 + 100]$ — this requires at most 101 darts.

After finding those points, we know that the center has to be inside a narrow crescent-shaped band of width 1. Intersecting that with the 101-by-101 square of candidates leaves at most 202 candidates, since there can't be more than 2 candidates that share an x value. Moreover, for most x values, the rounding ensures that there are less than 2, so the 199 darts we have left are enough to cover all candidates.

Notice that no complicated geometry is involved to find the candidates; the square is small enough to iterate over all points within it and check the distances to our hit and miss points to see if they are candidates or not.

Test Set 3

For Test Set 3, we can take a more typical geometric approach. To find the center of a circle, we can find 3 points on the edge of the circle, and then calculate a center from those. This has an issue: we only have nanometer precision, which means we would typically find points near the edge of the circle, but not exactly in it. The error that this introduces in our calculations could be significant. We can overcome this by bounding the error and then checking not only our

calculated center, but also other nearby points too. The math to calculate the center, and most importantly, to bound the error of it, can be hard and time consuming, but it is doable. In addition, you can wing it by not bounding the error and instead starting to throw at your calculated center and then spiral around it on nearby points until you find the actual center. This requires a leap of faith that you won't run out of tries, but it's a reasonable assumption. Fortunately, there is a simpler related approach that is more clearly correct, that we describe below. Of course, we still need to solve the problem of finding "almost-edge" points of the circle, which we also need for our simpler approach.

To find a point in the edge, we can't use the approach in Test Set 1, because if the dartboard's radius is not very large, those points could be really far away from the wall's edge. A different way of doing it is via a binary search. Say we know that point (x_0, y_0) is within the dartboard. Then, we know that for all x in the range $[-10^9, x_0]$ the points (x, y_0) are grouped such that there is a (possibly empty) interval having all misses, and then an interval having all hits. The same holds (in reverse) for $[x_0, 10^9]$. Analogously, points (x_0, y) for y in $[-10^9, y_0]$ or $[y_0, 10^9]$ are grouped by hits/misses. Then, for each of those ranges, we can do a binary search to find the switching point — that is, a point in the edge of the circle. This is exactly what we did in Test Set 2, except we fixed $x_0 = 0$, because the radius is so large that some points like (0, 0) are guaranteed to be inside the dartboard, and the range is small enough that we can scan it entirely instead of using binary search.

The searches above get us leftmost and rightmost points inside the dartboard for a given y-coordinate y_0, and topmost and bottommost points for a given x-coordinate x_0. Notice that the dartboard is symmetric with respect to the horizontal and vertical lines that go through its center. Therefore, the leftmost and rightmost points inside the dartboard in any fixed y-coordinate mirror each other, and the x-coordinate of the center is the midpoint between their x-coordinates. Similarly, we can find the y-coordinate of the center as the midpoint between the y-coordinates of the topmost and bottommost points inside the dartboard at any fixed x-coordinate.

The remaining task is to find a single point inside the dartboard. Since the area of the dartboard makes up a significant percentage of the wall area (at least $\pi / 16$), we could do this by throwing darts at random until we hit one point (the probability of hitting could be slightly less than $\pi / 16$ because we consider only points with integer nanometer coordinates, but the difference is negligible). This has an overwhelmingly high chance of hitting in 100 throws or fewer for all cases. However, there is a deterministic way that also works, which is to divide the wall into squares of side **A**. If the center of the dartboard is inside a square, it is guaranteed that at least one of the corners of that square is inside the dartboard. Therefore, we can simply try all those corners (there are only 25, and it is even possible to restrict it further), and we will find a hit point.

So, we need a fixed small number of darts (up to 100, depending on our method of choice) to find a point inside the dartboard, and each binary search needs at most $31 = \text{ceil}(\log_2 (2 \times 10^9 + 1))$. That is at most $4 \times 31 + 100$, which is a lot less than the 300 limit. In the first version, you need one fewer binary search — 3 edge points are enough to find a unique center — so after finding the center with error, you could have 150 darts left to explore its vicinity. Notice that if your initial point is actually one of the points (X - R, Y), (X + R, Y), (X, Y - R) or (X, Y + R), you would end up with two points instead of three and be unable to find the center. If we use the random procedure to find (x_0, y_0), the probability of this happening is negligible. Otherwise, we can detect the case and know that the two found points are opposite each other, so the center is their middle point.

Code Jam 2020 - Round 1B

Analysis: Expogo

Test Set 1

Test Set 1 is small enough to solve by hand. We can speed this up with a couple of observations:

- We can notice that every position with an even (X + Y) (apart from the origin) — hereafter an "even" position — seems to be unreachable. We can prove to ourselves that this is true: our initial X and Y coordinates of (0, 0) are both even, but only the first of our possible jumps (the 1-unit one) is of an odd length, and all jumps after that are of even lengths. So there is no way to reach any other "even" position starting from the origin, no matter how much jumping we do.
- We can find that all "odd" positions, on the other hand, can be reached using no more than 3 moves.
- To speed up solving the "odd" positions, we can take advantage of symmetry, as suggested in the explanation for Sample Case #2. For example, if we learn that `EEN` is a solution for (3, 4), then we also know that `WWS` is a solution for (-3, -4), and `EES` is a solution for (3, -4), and so on. Because of all the horizontal, vertical, and diagonal symmetry, there are really only six fundamentally different cases!
- We can check that our solutions are optimally short by using an argument like the one in the explanation for Sample Case #1. Any position with a Manhattan distance (that is, |**X**| + |**Y**|) of 1 cannot be reached in fewer than one jump; positions with Manhattan distances up to 3 and 7 require at least two or three jumps, respectively. If our solution lengths match these lower bounds — and they probably do unless we have jumped in an unusually indirect way — then they are valid.

Test Set 2

Based on the observations above, we may think to try a breadth-first search of all possible jumping paths, and continue until every "odd" (X, Y) position (with $-100 \leq X \leq 100$ and $-100 \leq Y \leq 100$) has been reached. It turns out that each such position is reachable in no more than 8 moves. We know that these solutions are optimally short because of the breadth-first nature of the search.

Test Set 3

Suppose that (**X**, **Y**) = (7, 10). In what direction should we make our initial 1-unit jump? As we saw above, we need our final X coordinate to be odd, but it is currently even, and we have only one chance to go from even to odd. Moving north or south will make our Y coordinate odd, but then we will never have another chance to make that even and the X coordinate odd. So we should either move west or east. For now, let's guess that we will go west; we will revisit the other possibility later.

That jump will take us to (-1, 0), and we will next need to make a 2-unit jump. Notice that we can make this look identical to the original problem setup, if we make two changes:

1. Shift (-1, 0) to be the new (0, 0). Then the goal becomes (8, 10) rather than (7, 10).
2. Transform the scale of the problem such that a 2-unit jump (to a "neighboring" cell) becomes the new 1-unit jump. Then the goal becomes (4, 5) instead of (8, 10).

With this in mind, let's revisit our original decision to jump to the west. If we had jumped east instead, we would have ended up at (1, 0), and if we had changed the problem in the same way we did above, our new goal would have been (3, 5). But that would be an "even" position (after rescaling), which cannot be reached! So we had no choice after all; we had to move west to be able to eventually reach the goal. It's a good thing we were so lucky!

So now the problem has "reset", and we are at (0, 0) and trying to get to (4, 5). In what direction should we make our "first" jump? Now we know we must move vertically, since 5 is odd and we will only have "one chance" to go from even to odd. If we jump north, the next rescaling will have a target of (2, 2), but if we jump south, the target will be (2, 3), which is the "odd" position that we want. From there, we should jump south to change the target to (1, 2), then east to change the target to (0, 1). At that point, we have a choice between jumping north and reaching the goal, and jumping south (which could still allow us to reach the goal after some further moves, e.g. one more to the south and then one more to the north). But the problem requires that we choose the shortest solution, so we should jump right to the goal! Therefore, the answer in this case is `WSSEN`.

Notice that this method is deterministic: we always have only one choice out of the four possible directions. We can rule out two of them because they will not make the correct coordinate odd. Of the other two, the new goal states they would leave us with must differ only in one of the coordinates and only by exactly 1 unit, and therefore one must be an "odd" position and the other must be an "even" position. It is possible that that "even" position is the goal, in which case we should jump there, but otherwise, we must choose the "odd" position.

The above analysis also shows that the only time we have a choice is when one of those options is to jump directly to the goal, in which case we obviously should. So we can be confident that our method produces the shortest possible solution. (We also know that that solution is unique, since if we were to choose to not jump directly to the goal when we had that option, we would only end up with a longer solution.)

Our method has a running time which is logarithmic in the magnitudes of the coordinates, so it will solve this test set blazingly fast!

A Code Jam callback!

This problem is a riff on the Pogo problem from Round 1C in 2013. If you were familiar with that problem, the analysis might have helped a bit with this one... but, like a well-designed pogo stick, Expogo is not *too* difficult to get a handle on anyway.

Code Jam 2020 - Round 1B

Analysis: Join the Ranks

Test Set 1

Test set 1 contains the 12 possible inputs allowed by the limits. We can try to solve the problem via a breadth-first search (BFS), on the graph of states, where a state is the current arrangement of cards in the deck. Notice, however, that cases with 14 total cards would yield an enormous graph and make the solution run too slowly — probably even too slowly for us to run the code locally to precompute answers that we can then hardcode!

However, one observation will help us: since the success condition does not involve the suits of the cards at all, we can ignore them and work only with the ranks. That dramatically cuts down the number of possible states, by going from (**R** × **S**)! to (**R** × **S**)! / ((**S**!)$^{\mathbf{R}}$). This allows a BFS to finish fast enough for this test set. We can also use this observation while solving the next test set...

Test Set 2

For test set 2, the worst case (**R**=40, **S**=40) has around 1.8 x 10^{2517} unique orderings. This means that the brute force solution will not work.

Our first important observation is that the reordering operation can decrease the number of adjacent cards of different ranks by at most two. In the starting configuration there are (**R** × **S**)-1 adjacent cards of different ranks. In the ending configuration there are **R**-1 adjacent cards of different ranks. So to get from (**R** × **S**)-1 to **R**-1 we need at least ceil((**R** × **S** - **R**) / 2) operations.

Now that we know ceil((**R** × **S** - **R**) / 2) is a lower bound on the answer, if we can come up with a method that is guaranteed to use no more than ceil((**R** × **S** - **R**) / 2) steps, then it will always produce a valid answer.

Now we will outline a way to sort the cards using exactly that many operations. The invariant we maintain is that at all times, for the ranks X and Y of any two consecutive cards, either Y = X, or Y = (X + 1) mod **R**. This is of course true for the initial ordering of the deck.

We repeatedly perform the following operation, as long as the number of adjacent pairs of cards of the same rank is less than **R** - 1 and the operation would not pick up the bottom card of the deck: find the largest block of consecutive cards from the top that contains exactly 2 different ranks to use as pile A. By the invariant, this will be one or more cards with rank X, followed by one or more cards with rank (X+1) mod **R**. Then, starting from the first card from the top that is not on pile A, take as pile B the largest block of consecutive cards that does not contain any cards of rank X, plus all consecutive cards of rank X that immediately follow that block. Notice that at least one such card of rank X must exist; otherwise, by the invariant, the number of adjacent pairs of cards of different ranks would already be **R**-1.

We can show that this operation reduces the number of adjacent cards of different ranks by 2 every time it does not pick up the bottom card of the deck. To show this, notice that the bottom of pile B is a card of rank X and the first card left over in the deck is, by the invariant, (X + 1) mod **R**. That means that the new adjacent pairs are two cards of rank X (the bottom of pile B and the top of pile A) and two cards of rank (X + 1) mod **R** (the bottom of pile A and the top of the leftover deck). The broken adjacent pairs are — by definition of piles A and B — both of cards of different rank. Therefore, the number of adjacent pairs of cards of different rank decreases by 2 with this operation.

Suppose that performing the operation would pick up the bottom card of the deck. That means that all cards of rank X are in two contiguous blocks at the top and bottom of the deck before the operation is performed. In addition, since this is the first time the bottom card of the deck is picked up for an operation, X = **R**. Because of the invariant, that requires every other rank to be in a single contiguous block. In this ordering, there are exactly **R** adjacent pairs of cards with different ranks. Instead of the operation above, we finish by making pile A consist of the largest block of consecutive cards of rank **R**, starting at the top, and pile B be the rest of the deck. After performing the operation, **R** - 1 pairs of adjacent cards of different ranks remain (by a similar argument as before, ignoring the broken and created pairs that involved the leftover deck, since there is none leftover) and the final card of the deck is still **R**.

After performing the repeated operation floor((**R** × **S** - **R**) / 2) times, the number of adjacent pairs of cards of the same rank decreases to **R** - 1 if **R** × **S** - **R** is even or **R** if it's odd. Notice that the number remains greater than **R** before each such operation, so we would never have picked up the bottom card of the deck. In the even case, the number of adjacent pairs of cards of the same rank is now minimum and we never picked up the bottom card of the deck, so we are at exactly the target ordering. In the odd case, we arrive at the case in which we do pick up the bottom card of the deck with our last operation, but as argued above, that operation also leaves the deck in the target order.